University of Kalyani

Computer Science & Engineering

Name : Aryan Karan

Course: B.Tech.

Regd. No. : 1090018 OF 2023-24

Roll No. : 90/BCS/230014

Session: 2024-25

Class Roll No.: 31

Subject: CS-391

Semester: 3rd

Assignment - 1

Stack using array

```c
#include <stdio.h>
#include <stdbool.h>
#define SIZE 4


int top=-1 , stack[SIZE];


void push(int n)
{

    if (top >= SIZE-1 )
        printf("Stack is full\n");
    else
        {
            top++;
            stack[top]=n;
        }
}


void pop()
{
    int n;
    if (top < 0 )
        printf("Stack is empty\n");
    else
        {
            n=stack[top];
            top--;
            printf("Popped value: %d\n\n", n);
        }
}


void display()
{
    if (top < 0)
        printf("No element to display\n");
    else
        {
```

```
            for (int i=top ; i>=0 ; i--)
                printf("%d ", stack[i]);
            printf("\n");
        }
}


int main()
{

    while (true)
    {
        // Enter choices
        printf("\nEnter a choice :\n");
        printf("1. Push\n");
        printf("2. Pop an element\n");
        printf("3. Display the stack\n");
        printf("4. Exit menu\n");

        int choice;
        printf("Enter a choice: ");
        scanf("%d", &choice);
        printf("\n");

        int n;
        switch (choice)
        {

            case 1:
                printf("Enter an integer: ");
                scanf("%d", &n);
                push(n);
                break;

            case 2:
                pop();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("whoops! Invalid input\n");
        }
```

```
    }
}
```

OUTPUT :->

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 1

Enter an integer: 43

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 1

Enter an integer: 56

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 1

Enter an integer: 45

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 3

45 56 43

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 2

Popped value: 45

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 3

56 43

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 2

Popped value: 56

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice: 3

43

Enter a choice :

1. Push

2. Pop an element

3. Display the stack

4. Exit menu

Enter a choice:

Assignment 2

Queue using array

```c
#include <stdio.h>
#define SIZE 2

int queue[SIZE];
int front=-1, rear=-1, choice, n;

int main()
{
    while (1)
    {
        printf("1. Insert an element. \n2. Delete an element.\n3. Display
queue. \n4. Exit\n\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        // check choice
        switch (choice)
        {
            case 1: // insert
                if (rear >= SIZE-1)
                    printf("Queue is full\n");
                else
                    {
                        rear++;
                        printf("Enter element: ");
                        scanf("%d", &n);
                        printf("\n");
                        queue[rear]=n;

                        if (front == -1 )
                            front = 0;
                    }
                break;


            case 2: // remove
                if (front == -1)
                    printf("queue is empty\n");
                else
                {
                    n=queue[front];
                    printf("Removed element %d\n", n);
```

```
                    if ( front != rear )
                        front++;
                    else
                        {
                            front = -1;
                            rear = -1;
                            // printf("DEBUG:  Done empty condition!\n");
                        }
                }
                break;


            case 3: // Display operation
                if (front == -1)
                    printf("Queue is empty\n");
                else
                {
                    for (int i=front ; i <= rear ; i++)
                        printf("%d ", queue[i]);
                        printf("\n");
                }
                break;


            case 4: // exit from program
                return 0;

            default:
                printf("Invalid Choice. \n");
        }
    }
}
```

OUTPUT =>

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 1

Enter element: 43

1. Insert an element.
2. Delete an element.
3. Display queue.
4. Exit

Enter your choice: 1

Enter element: 34

1. Insert an element.
2. Delete an element.
3. Display queue.
4. Exit

Enter your choice: 1

Queue is full

1. Insert an element.
2. Delete an element.
3. Display queue.
4. Exit

Enter your choice: 3

43 34

1. Insert an element.
2. Delete an element.
3. Display queue.
4. Exit

Enter your choice: 2

Removed element 43

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 3

34

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice:

Assignment 3

Stack using Linked list

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;

void push(int n) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed. Unable to push.\n");
        return;
    }

    newNode->data = n;
    newNode->next = top;  // Link new node to the current top
    top = newNode;        // Update top pointer to new node
    printf("%d pushed to stack.\n", n);
}

void pop() {
    if (top == NULL) {
        printf("Stack is empty. Nothing to pop.\n");
        return;
    }

    struct Node* temp = top;
    int poppedValue = temp->data;
    top = top->next;  // Update top to the next node
    free(temp);

    printf("Popped value: %d\n", poppedValue);
}

int peek() {
    if (top == NULL) {
        printf("Stack is empty. No top element.\n");
        return -1;  // Indicates an empty stack
```

```
    }
    return top->data;
}

bool isEmpty() {
    return (top == NULL);
}


void display() {
    if (top == NULL) {
        printf("Stack is empty. Nothing to display.\n");
        return;
    }

    printf("Stack elements (top to bottom): ");

    struct Node* current = top;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}


int main() {
    int choice, value;
    while (true) {
        printf("\n==== Stack Menu ====\n");
        printf("1. Push an element\n");
        printf("2. Pop an element\n");
        printf("3. Display stack\n");
        printf("4. Peek at top element\n");
        printf("5. Check if stack is empty\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        printf("\n");

        switch (choice) {
            case 1:
                printf("Enter an integer to push: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
```

```
                pop();
                break;

            case 3:
                display();
                break;

            case 4:
                value = peek();
                if (value != -1)
                    printf("Top element is: %d\n", value);
                break;

            case 5:
                printf(isEmpty() ? "The stack is empty.\n" : "The stack is not
empty.\n");
                break;

            case 6:
                printf("Exiting the menu. Goodbye!\n");
                exit(0);

            default:
                printf("Invalid choice. Please enter a valid option.\n");
        }
    }
    return 0;
}
```

Output :->

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 1

Enter an integer to push: 54

54 pushed to stack.

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 1

Enter an integer to push: 34

34 pushed to stack.

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 1

Enter an integer to push: 87

87 pushed to stack.

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 3

Stack elements (top to bottom): 87 34 54

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 2

Popped value: 87

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 4

Top element is: 34

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 5

The stack is not empty.

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 2

Popped value: 34

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 2

Popped value: 54

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 2

Stack is empty. Nothing to pop.

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 5

The stack is empty.

==== Stack Menu ====

1. Push an element

2. Pop an element

3. Display stack

4. Peek at top element

5. Check if stack is empty

6. Exit

Enter your choice: 6

Exiting the menu. Goodbye!

Assignment 4

Queue using linked list

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

void enqueue(int n) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    newNode->data = n;
    newNode->next = NULL;
    if (rear == NULL) { // Queue is empty
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }

    printf("Inserted element: %d\n", n);
}

void dequeue() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    struct Node* temp = front;
    int removed = temp->data;
    front = front->next;
    if (front == NULL) { // Queue becomes empty
        rear = NULL;
    }
```

```
    free(temp);
    printf("Removed element: %d\n", removed);
}

void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    struct Node* current = front;
    printf("Queue: ");
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

int main() {
    int choice, n;
    while (1) {
        printf("\n1. Insert an element.\n2. Delete an element.\n3. Display
queue.\n4. Exit\n\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter element: ");
                scanf("%d", &n);
                enqueue(n);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
```

```
                printf("Invalid Choice.\n");
        }
    }
    return 0;
}
```

Output :->

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 1

Enter element: 43

Inserted element: 43

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 1

Enter element: 65

Inserted element: 65

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 1

Enter element: 12

Inserted element: 12

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 1

Enter element: 87

Inserted element: 87

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 3

Queue: 43 65 12 87

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 2

Removed element: 43

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 3

Queue: 65 12 87

1. Insert an element.

2. Delete an element.

3. Display queue.

4. Exit

Enter your choice: 4

Assignment 5

Singly Linked list

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

typedef struct Node *Nodeptr;
typedef struct Node list;
typedef list Node;


Nodeptr getnode() {
    Nodeptr head;
    head = (Nodeptr)malloc(sizeof(struct Node));
    return head;
}


Nodeptr InsertBegin(list* head) {
    Nodeptr p = getnode();
    // Data
    printf("Enter data: ");
    scanf("%d", &p->data);

    p->next = NULL;
    if (head != NULL) {
        p->next = head;
        head = p;
    } else {
        head = p;
    }

    return head;
}

void InsertEnd(list* head) {
    Nodeptr p = getnode();
    // Data
    printf("Enter data: ");
    scanf("%d", &p->data);
```

```
    Nodeptr q = head;
    while (q->next != NULL) {
        q = q->next;
    }

    q->next = p;
    p->next = NULL;
}


Nodeptr Insert_any_position(list* head, int pos) {
    if (pos == 1 ) {
        head = InsertBegin(head);
        return head;
    }

    Nodeptr p, q, r;

    q = head->next;
    while (--pos && q != NULL) {
        r = q;
        q = q->next;
    }

    if (q != NULL){

        // Get data only if in between
        p = getnode();
        p->next = q;
        printf("Enter Data: ");
        scanf("%d", &p->data);

        r->next = p;
    } else {
        // Check out of list , i.e beyond list as q != NULL checked
        if (pos > 0) {
            printf("Out of list !!!\n");
        } else {
            InsertEnd(head);
        }
    }

    return head;
}


/*
Nodeptr InsertBetween(Nodeptr head, int pos) {
    Nodeptr p = getnode();
```

```
    // Data
    scanf("%d", &p->data);

    Nodeptr q = head;

    while (--pos && q != NULL) {
        q = q->next;
    }
}
*/

list *createlist() {
    list *head = NULL, *p, *q;
    int choice = 1;
    while (choice == 1) {
        p = getnode();
        printf("Enter data: ");
        scanf("%d", &p->data);
        p->next = NULL;
        if (head == NULL) {
            head = p;
            q = p;
        } else {
            q->next = p;
            q = p;
        }
        printf("Press 1 to enter next data or 2 to exit: ");
        scanf("%d", &choice);
    }
    return head;
}



void displaylist(list* head) {
    if (head == NULL) {
        printf("Empty list\n");
    } else {
        list *p = head;
        while (p != NULL) {
            printf("%d->", p->data);
            p = p->next;
        }
        printf("NULL\n");
    }
}
```

```c
list* deleteEndNode(list* head) {
    Nodeptr p = head;
    Nodeptr r;
    while (p->next != NULL) {
        r = p;
        p = p->next;
    }
    r->next = NULL;
    free(p);
}

Nodeptr delBegin(Nodeptr head) {
    Nodeptr p = head;
    head = p->next;

    // Display data being deleted
    printf("Deleting data: %d\n", p->data);
    free(p);

    return head;
}

Nodeptr delNode(Nodeptr head, int pos) {
    list *p = head, *r;

    if (pos == 1) {
        head = delBegin(head);
        return head;
    }

    while (--pos && p != NULL) {
        r = p;
        p = p->next;
    }

    if (p != NULL) {
        r->next = p->next;

        // Display deleting data
        printf("Deleting data: %d\n", p->data);
        free(p);
    }
    return head;
}


list* concatList(list* head1, list* head2) {
    if (head1 != NULL && head2 != NULL) {
        Nodeptr p = head1;
```

```
        while (p->next != NULL) {
            p = p->next;
        }

        p->next = head2;
        return head1;
    }

    if (head2 != NULL) {
        return head2;
    }

    if (head2 == NULL) {
        printf("Both lists empty!\n");
        return head1;
    }

}


list* bubbleSort(list* head) {
    for (list* p = head; p->next != NULL; p = p->next) {
        for (list* q = p->next; q != NULL; q = q->next) {
            if (p->data > q->data) {
                //swap
                int temp = p->data;
                p->data = q ->data;
                q->data = temp;
            }
        }
    }
    return head;
}



list* reverseList(list* head) {
    Nodeptr p, q, r;
    p = head;
    q = NULL;

    while (p != NULL) {
        r = q;
        q = p;
        p = p->next;
        q->next = r;
    }
```

```
    head = q;
    return head;
}


list* linearSearch(list* head, int n) {
    list* p = head;
    int counter = 1;
    while (p != NULL) {
        if (p->data == n) {
            printf("Found %d at position: %d\n", n, counter);
            break;
        } else {
            p = p->next;
            counter++;
        }
    }

    if (p == NULL) {
        printf("Not found\n");
    }

    return head;
}


int main() {
    list *head = NULL;
    int choice, pos;
    while (1) {
        printf("1. Create list \n2. Display list \n3. Insert between \n4.
Delete a certain node \n5. Concatenate two lists \n6. Sort \n7. Reverse List
\n8. Search \n9. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                head = createlist();
                break;

            case 2:
                displaylist(head);
                break;

            case 3:
                printf("Enter the postition: ");
                scanf("%d", &pos);

                head = Insert_any_position(head, pos);
```

```
                break;
/*
            case 4:
                head = InsertBegin(head);
                break;

            case 5:
                head = InsertEnd(head);
                break;
*/


            case 4:
                // Get node position
                printf("Enter node position to delete: ");
                scanf("%d", &pos);

                printf("Original List: ");
                displaylist(head);
                head = delNode(head, pos);
                printf("Updated List: ");
                displaylist(head);
                break;

            case 5:
                printf("Enter 1st List data:-> \n");
                list* head1 = createlist();
                printf("Enter 2nd List data:-> \n");
                list* head2 = createlist();

                printf("List 1: ");
                displaylist(head1);

                printf("List 2: ");
                displaylist(head2);

                head = concatList(head1, head2);
                printf("Combined list: ");
                displaylist(head);
                break;

            case 6:
                printf("Original List: ");
                displaylist(head);

                head = bubbleSort(head);
                printf("Sorted   List: ");
                displaylist(head);
                break;
```

```
            case 7:
                printf("Original List: ");
                displaylist(head);

                head = reverseList(head);

                printf("Reversed List: ");
                displaylist(head);
                break;

            case 8:
                int n;
                printf("eNter data to search: ");
                scanf("%d", &n);

                linearSearch(head, n);
                break;

            case 9:
                return 0;

            default:
                printf("Invalid choice !!\n");
        }
    }
}
```

OutPut :->

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 1

Enter data: 54

Press 1 to enter next data or 2 to exit: 1

Enter data: 34

Press 1 to enter next data or 2 to exit: 1

Enter data: 76

Press 1 to enter next data or 2 to exit: 1

Enter data: 23

Press 1 to enter next data or 2 to exit: 1

Enter data: 76

Press 1 to enter next data or 2 to exit: 2

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 2

54->34->76->23->76->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 3

Enter the postition: 4

Enter Data: 45

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 3

Enter the postition: 1

Enter data: 54

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 2

54->54->34->76->23->45->76->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 4

Enter node position to delete: 5

Original List: 54->54->34->76->23->45->76->NULL

Deleting data: 23

Updated List: 54->54->34->76->45->76->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 6

Original List: 54->54->34->76->45->76->NULL

Sorted List: 34->45->54->54->76->76->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 7

Original List: 34->45->54->54->76->76->NULL

Reversed List: 76->76->54->54->45->34->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 8

eNter data to search: 45

Found 45 at position: 5

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 8

eNter data to search: 23

Not found

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 7

Original List: 76->76->54->54->45->34->NULL

Reversed List: 34->45->54->54->76->76->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 5

Enter 1st List data:->

Enter data: 12

Press 1 to enter next data or 2 to exit: 1

Enter data: 65

Press 1 to enter next data or 2 to exit: 1

Enter data: 23

Press 1 to enter next data or 2 to exit: 1

Enter data: 23

Press 1 to enter next data or 2 to exit: 2

Enter 2nd List data:->

Enter data: 65

Press 1 to enter next data or 2 to exit: 1

Enter data: 23

Press 1 to enter next data or 2 to exit: 1

Enter data: 76

Press 1 to enter next data or 2 to exit: 2

List 1: 12->65->23->23->NULL

List 2: 65->23->76->NULL

Combined list: 12->65->23->23->65->23->76->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 6

Original List: 12->65->23->23->65->23->76->NULL

Sorted List: 12->23->23->23->65->65->76->NULL

1. Create list

2. Display list

3. Insert between

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice:

Assignment 6

Circular Linked List

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

typedef struct Node* Nodeptr;
typedef struct Node list;


Nodeptr getnode() {
    Nodeptr head;
    head = (Nodeptr)malloc(sizeof(struct Node));
    return head;
}


Nodeptr InsertBegin(list * head) {
    Nodeptr p = getnode();
    // Data
    printf("Enter data: ");
    scanf("%d", &p->data);

    if (head != NULL) {
        p->next = head->next;
        head->next = p;
    } else {
        head = p;
        p->next = p;
    }

    return head;
}


Nodeptr InsertEnd(list* head) {
    head = InsertBegin(head);
    head = head->next;

    return head;
}
```

```c
Nodeptr Insert_any_position(list* head, int pos) {
    if (pos == 1 ) {
        head = InsertBegin(head);
        return head;
    }

    Nodeptr p, q, r;


    q = head->next;
    while (--pos && q != head) {
        r = q;
        q = q->next;
    }

    if (q != head){
        p->next = q;

        // Get data only if in between
        p = getnode();
        printf("Enter Data: ");
        scanf("%d", &p->data);

        r->next = p;
    } else {
        // Check out of list , i.e beyond head as q != head checked
        if (pos > 0) {
            printf("Out of list !!!\n");
        } else {
            head = InsertEnd(head);
        }
    }

    return head;
}


Nodeptr InsertBetween(Nodeptr head, int pos) {
    Nodeptr p = getnode();
    // Data
    scanf("%d", &p->data);

    Nodeptr q = head;

    while (--pos && q != NULL) {
        q = q->next;
    }
```

```
}


list *createlist() {
    Nodeptr head = NULL, p, q;
    int choice = 1;
    while (choice == 1) {
        p = getnode();
        printf("Enter data: ");
        scanf("%d", &p->data);

        if (head == NULL) {
            head = p;
            q = p;
            p->next = p;
        } else {
            q->next = p;
            q = p;
            q->next = head;
        }
        printf("Press 1 to enter next data or 2 to exit: ");
        scanf("%d", &choice);
    }

    head = q;
    return head;
}


void displaylist(list* head) {
    if (head == NULL) {
        printf("Empty list\n");
    } else {
        list *p = head->next;
        while (p != head) {
            printf("%d->", p->data);
            p = p->next;
        }
        printf("%d--->%d\n", head->data, head->next->data);
        // printf("%d-\n↑---⮌\n", head->data);
    }
}




list* deleteEndNode(list* head) {
    Nodeptr p = head;
    Nodeptr r;
```

```
    while (p->next != NULL) {
        r = p;
        p = p->next;
    }
    r->next = NULL;
    free(p);
}

Nodeptr delBegin(Nodeptr head) {
    Nodeptr p = head;

}

Nodeptr delNode(Nodeptr head, int pos) {
    list * p = head, *r;

/* TODO: delBegin function
    if (pos == 1) {
        head = deleteBegin(head);
        return head;
    }
*/

    while (--pos && p != NULL) {
        r = p;
        p = p->next;
    }

    if (p != NULL) {
        r->next = p->next;

        // Display deleting data
        printf("Deleting data: %d\n", p->data);
        free(p);
    }
    return head;
}


list* concatList(list* head1, list* head2) {
    if (head1 != NULL && head2 != NULL) {

        Nodeptr p = head1->next;
        head1->next = head2->next;
        head2->next = p;

        return head2;
    }
/*
```

```
        while (p->next != NULL) {
            p = p->next;
        }

        p->next = head2;
        return head1;

    }
*/


    if (head2 != NULL) {
        return head2;
    }

    if (head2 == NULL) {
        printf("Both lists empty!\n");
        return head1;
    }
}


list* bubbleSort(list* head) {
    for (list* p = head; p->next != NULL; p = p->next) {
        for (list* q = p->next; q != NULL; q = q->next) {
            if (p->data > q->data) {
                //swap
                int temp = p->data;
                p->data = q ->data;
                q->data = temp;
            }
        }
    }
    return head;
}



list* reverseList(list* head) {
    Nodeptr p, q, r;
    p = head;
    q = NULL;

    while (p != NULL) {
        r = q;
        q = p;
        p = p->next;
        q->next = r;
    }
```

```
    head = q;
    return head;
}


list* linearSearch(list* head, int n) {
    list* p = head;
    int counter = 1;
    while (p != NULL) {
        if (p->data == n) {
            printf("Found %d at position: %d\n", n, counter);
            break;
        } else {
            p = p->next;
            counter++;
        }
    }

    if (p == NULL) {
        printf("Not found\n");
    }

    return head;
}


int main() {
    list *head = NULL;
    int choice, pos;
    while (1) {
        printf("1. Create list\n2. Display list\n3. Insert at Beginning \n4.
Insert at End \n5. Insert between \n6. Delete a certain node \n7. Concatenate
two lists \n8. Sort \n9. Reverse List \n10. Search \n11. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                head = createlist();
                break;

            case 2:
                displaylist(head);
                break;

            case 3:
                head = InsertBegin(head);
                break;
```

```
            case 4:
                head = InsertEnd(head);
                break;

            case 5:
                printf("Enter the postition: ");
                scanf("%d", &pos);

                head = Insert_any_position(head, pos);
                break;

            case 6:
                // Get node position
                printf("Enter node position to delete: ");
                scanf("%d", &pos);

                head = delNode(head, pos);
                printf("Updated List: ");
                displaylist(head);
                break;

            case 7:
                printf("Enter 1st List data:-> \n");
                list* head1 = createlist();
                printf("Enter 2nd List data:-> \n");
                list* head2 = createlist();

                printf("List 1: ");
                displaylist(head1);

                printf("List 2: ");
                displaylist(head2);

                head = concatList(head1, head2);
                printf("Combined list: ");
                displaylist(head);
                break;

            case 8:
                head = bubbleSort(head);
                displaylist(head);
                break;

            case 9:
                head = reverseList(head);
                printf("Reversed List: ");
                displaylist(head);
                break;
```

```
            case 10:
                int n;
                printf("eNter data to search: ");
                scanf("%d", &n);

                linearSearch(head, n);
                break;

            case 11:
                return 0;

            default:
                printf("Invalid choice !!\n");
        }
    }
}


/*
Puja HW :->
1. Sort a circular linked list
2. Searching an element
3. reversing c.l.l.
*/
```

OUTPUT :->

1. Create list
2. Display list
3. Insert at Beginning
4. Insert at End
5. Insert between
6. Delete a certain node
7. Concatenate two lists
8. Sort
9. Reverse List
10. Search
11. Exit

Enter your choice: 1

Enter data: 54

Press 1 to enter next data or 2 to exit: 1

Enter data: 45

Press 1 to enter next data or 2 to exit: 1

Enter data: 34

Press 1 to enter next data or 2 to exit: 1

Enter data: 54

Press 1 to enter next data or 2 to exit: 1

Enter data: 78

Press 1 to enter next data or 2 to exit: 2

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 2

54->45->34->54->78--->54

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 3

Enter data: 54

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 2

54->54->45->34->54->78--->54

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 4

Enter data: 89

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 2

54->54->45->34->54->78->89--->54

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 6

Enter node position to delete: 3

Deleting data: 54

Updated List: 54->45->34->54->78->89--->54

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 2

54->45->34->54->78->89--->54

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 9

Reversed List: 78->54->34->45->54->89--->78

1. Create list

2. Display list

3. Insert at Beginning

4. Insert at End

5. Insert between

6. Delete a certain node

7. Concatenate two lists

8. Sort

9. Reverse List

10. Search

11. Exit

Enter your choice: 10

eNter data to search: 34

Found 34 at position: 4

Assignment 7

Doubly Linked List

```c
#include <stdio.h>
#include <stdlib.h>

/*
sruct Node {
    int data;
    struct Node *next;
};
*/

struct Node {
    struct Node* previous;
    int data;
    struct Node* next;
};

typedef struct Node* Nodeptr;
typedef struct Node dblist;
typedef struct Node list;

Nodeptr getnode() {
    Nodeptr head;
    head = (Nodeptr)malloc(sizeof(dblist));
    return head;
}

Nodeptr InsertBegin(dblist* head) {
    Nodeptr p = getnode();
    // Data
    printf("Enter data: ");
    scanf("%d", &p->data);

    p->previous = NULL;
    if (head != NULL) {
        head->previous = p;
        p->next = head;
        head = p;
    } else {
        head = p;
        return head;
    }
}
```

```c
Nodeptr InsertEnd(dblist* head) {
    Nodeptr p = getnode(), q;
    // Data
    printf("Enter data: ");
    scanf("%d", &p->data);
    p->next = NULL;

    q = head;

    if (head != NULL) {
        // traverse to end
        while (q->next != NULL) {
            q = q->next;
        }

        // attach at the end
        q->next = p;
        p->previous = q;

    return head;

    } else {
        head = p;
        return head;
    }
}


/*
Nodeptr InsertAnyPos(Nodeptr head, int pos) {
    Nodeptr p = getnode();
    // Data
    scanf("%d", &p->data);

    Nodeptr q = head;

    while (--pos && q != NULL) {
        q = q->next;
    }
}
*/


Nodeptr Insert_any_position(list* head, int pos) {
    if (pos == 1 ) {
        head = InsertBegin(head);
        return head;
    }
```

```
    Nodeptr p = getnode(), q, r;

    // no dangling pointers
    p->next = NULL;
    p->previous = NULL;

    if (head == NULL && pos != 1) {
        printf("Empty List & inserting beyond first pos !!!\n");
        return head;
    }

    q = head;
    // r = q;
    while (--pos && q != NULL) {
        r = q;
        q = q->next;
    }


    if (q != NULL){
        p->next = q;
        p->previous = r;
        r->next = p;
        q->previous = p;

        // Get data only if in between
        printf("Enter Data: ");
        scanf("%d", &p->data);

    } else {
        // Check out of list , i.e beyond head as q != head checked
        if (pos > 0) {
            printf("Out of list !!!\n");
        } else {
            head = InsertEnd(head);
        }
    }

    return head;
}



list* createlist() {
    Nodeptr head = NULL, p, q;
    int choice = 1;
    while (choice == 1) {
        p = getnode();
```

```
        printf("Enter data: ");
        scanf("%d", &p->data);
        p->next = NULL;

        if (head == NULL) {
            head = p;
            q = p;
            head->previous = NULL;
        } else {
            q->next = p;
            p->previous = q;
            q = p;
        }

        printf("Press 1 to enter next data or 2 to exit: ");
        scanf("%d", &choice);
    }
    return head;
}

void displaylist(list* head) {
    if (head == NULL) {
        printf("Empty list\n");
    } else {
        list *p = head, *q;

        printf("Forward: ");
        while (p != NULL) {
            printf("%d->", p->data);
            q = p;
            p = p->next;
        }
        printf("NULL\n");

        printf("Reverse: ");
        while (q != NULL) {
            printf("%d->", q->data);
            q = q->previous;
        }
        printf("NULL\n");
    }
}


// Redundant
list* deleteEndNode(list* head) {
    Nodeptr p = head;
    Nodeptr r;
    while (p->next != NULL) {
```

```
        r = p;
        p = p->next;
    }
    r->next = NULL;
    free(p);
}



Nodeptr delBegin(Nodeptr head) {
    Nodeptr p = head;
    head = p->next;
    head->previous = NULL;

    // Display data being deleted
    printf("Deleting data: %d\n", p->data);
    free(p);

    return head;
}


Nodeptr delNode(Nodeptr head, int pos) {
    list *p = head, *r;

    if (pos == 1) {
        head = delBegin(head);
        return head;
    }

    while (--pos && p != NULL) {
        r = p;
        p = p->next;
    }

    if (p != NULL) {
        r->next = p->next;
        (p->next)->previous = r;

        // Display deleting data
        printf("Deleting data: %d\n", p->data);
        free(p);
    }
    return head;
}
```

```
// Single :->

// Nodeptr delNode(Nodeptr head, int pos) {
//     list *p = head, *r;

//     /* TODO: delBegin function
//         if (pos == 1) {
//             head = deleteBegin(head);
//             return head;
//         }
//     */

//     while (--pos && p != NULL) {
//         r = p;
//         p = p->next;
//     }

//     if (p != NULL) {
//         r->next = p->next;

//         // Display deleting data
//         printf("Deleting data: %d\n", p->data);
//         free(p);
//     }
//     return head;
// }



list* concatList(list* head1, list* head2) {
    if (head1 != NULL && head2 != NULL) {
        Nodeptr p = head1;

        while (p->next != NULL) {
            p = p->next;
        }

        p->next = head2;
        head2->previous = p;
        return head1;
    }

    if (head2 != NULL) {
        return head2;
    }

    if (head2 == NULL) {
        printf("Both lists empty!\n");
        return head1;
```

```
    }
}

list* bubbleSort(list* head) {
    for (list* p = head; p->next != NULL; p = p->next) {
        for (list* q = p->next; q != NULL; q = q->next) {
            if (p->data > q->data) {
                // swap
                int temp = p->data;
                p->data = q->data;
                q->data = temp;
            }
        }
    }
    return head;
}

list* reverseList(list* head) {
    Nodeptr p, q, r;
    p = head;
    q = NULL;

    while (p != NULL) {
        r = q;
        q = p;
        p = p->next;
        q->next = r;
    }

    head = q;
    return head;
}

list* linearSearch(list* head, int n) {
    list* p = head;
    int counter = 1;
    while (p != NULL) {
        if (p->data == n) {
            printf("Found %d at position: %d\n", n, counter);
            break;
        } else {
            p = p->next;
            counter++;
        }
    }

    if (p == NULL) {
        printf("Not found\n");
```

```
    }

    return head;
}

int guard(Nodeptr head) {
    if (head == NULL) {
        printf("Can't operate on an empty list !!! \n");
        return 1;
    }
    return 0;
}


int main() {
    list* head = NULL;
    int choice, pos;
    while (1) {
        printf("1. Create list\n2. Display list\n3. Insert at any postion \n4.
Delete a certain node \n5. Concatenate two lists \n6. Sort \n7. Reverse List
\n8. Search \n9. Exit\n");
/*
4. Insert at beginning\n5. Insert at end \n
*/
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                head = createlist();
                break;

            case 2:
                displaylist(head);
                break;
/*
            case 4:
                head = InsertBegin(head);
                break;

            case 5:
                head = InsertEnd(head);
                break;
*/
            case 3:
                printf("Enter the postition: ");
                scanf("%d", &pos);

                head = Insert_any_position(head, pos);
                break;
```

```
        case 4:
            if ( guard(head) )  break;

            // Get node position
            printf("Enter node position to delete: ");
            scanf("%d", &pos);

            printf("Original List:-> \n");
            displaylist(head);
            head = delNode(head, pos);
            printf("Updated List:-> \n");
            displaylist(head);
            break;

        case 5:
            printf("Enter 1st List data:-> \n");
            list* head1 = createlist();
            printf("Enter 2nd List data:-> \n");
            list* head2 = createlist();

            printf("List 1:-> \n");
            displaylist(head1);

            printf("List 2:-> \n");
            displaylist(head2);

            head = concatList(head1, head2);
            printf("Combined list:-> \n");
            displaylist(head);
            break;

        case 6:
            if ( guard(head) )  break;

            head = bubbleSort(head);
            displaylist(head);
            break;

        case 7:
            if ( guard(head) )  break;

            head = reverseList(head);
            printf("Reversed List:-> \n");
            displaylist(head);
            break;

        case 8:
```

```
                if ( guard(head) )  break;

                int n;
                printf("eNter data to search: ");
                scanf("%d", &n);

                linearSearch(head, n);
                break;

            case 9:
                return 0;

            default:
                printf("Invalid choice !!\n");
        }
    }
}



/*
- DLL done ☑
- Circular DLL

- Insert
- Delete
- Concat

- Circular DLL

Array :->
- Quick sort
Merge Sort
Radix sort


Tree :-
- Insert
- Display, Inorder : usccesssor, Preorder, postorder, infix, postfix
- delete node from tree : child (left, right)

*/
```

OUTPUT :->

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 1

Enter data: 43

Press 1 to enter next data or 2 to exit: 1

Enter data: 23

Press 1 to enter next data or 2 to exit: 1

Enter data: 65

Press 1 to enter next data or 2 to exit: 1

Enter data: 21

Press 1 to enter next data or 2 to exit: 1

Enter data: 76

Press 1 to enter next data or 2 to exit: 2

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 2

Forward: 43->23->65->21->76->NULL

Reverse: 76->21->65->23->43->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 3

Enter the postition: 5

Enter Data: 12

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 2

Forward: 43->23->65->21->12->76->NULL

Reverse: 76->12->21->65->23->43->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 4

Enter node position to delete: 3

Original List:->

Forward: 43->23->65->21->12->76->NULL

Reverse: 76->12->21->65->23->43->NULL

Deleting data: 65

Updated List:->

Forward: 43->23->21->12->76->NULL

Reverse: 76->12->21->23->43->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 4

Enter node position to delete: 68

Original List:->

Forward: 43->23->21->12->76->NULL

Reverse: 76->12->21->23->43->NULL

Updated List:->

Forward: 43->23->21->12->76->NULL

Reverse: 76->12->21->23->43->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 6

Forward: 12->21->23->43->76->NULL

Reverse: 76->43->23->21->12->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 7

Reversed List:->

Forward: 76->43->23->21->12->NULL

Reverse: 12->21->23->43->76->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 8

eNter data to search: 43

Found 43 at position: 2

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 2

Forward: 76->43->23->21->12->NULL

Reverse: 12->21->23->43->76->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 5

Enter 1st List data:->

Enter data: 12

Press 1 to enter next data or 2 to exit: 1

Enter data: 65

Press 1 to enter next data or 2 to exit: 1

Enter data: 34

Press 1 to enter next data or 2 to exit: 1

Enter data: 76

Press 1 to enter next data or 2 to exit: 2

Enter 2nd List data:->

Enter data: 23

Press 1 to enter next data or 2 to exit: 1

Enter data: 5

Press 1 to enter next data or 2 to exit: 2

List 1:->

Forward: 12->65->34->76->NULL

Reverse: 76->34->65->12->NULL

List 2:->

Forward: 23->5->NULL

Reverse: 5->23->NULL

Combined list:->

Forward: 12->65->34->76->23->5->NULL

Reverse: 5->23->76->34->65->12->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 2

Forward: 12->65->34->76->23->5->NULL

Reverse: 5->23->76->34->65->12->NULL

1. Create list

2. Display list

3. Insert at any postion

4. Delete a certain node

5. Concatenate two lists

6. Sort

7. Reverse List

8. Search

9. Exit

Enter your choice: 9

Assignment 8

Binary Search Tree

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    struct Node* left;
    int data;
    struct Node* right;
};

typedef struct Node Tree;
typedef struct Node* Nodeptr;

Nodeptr getnode() {
    Nodeptr head;
    head = (Nodeptr)malloc(sizeof(struct Node));  // Fixed allocation size (was sizeof(Nodeptr))
    // head->data = input_data;
    return head;
}

Nodeptr InsertBST(Nodeptr Tree, Nodeptr z) {
    // Nodeptr Tree = NULL;
    Nodeptr y = NULL;
    Nodeptr x = Tree;

    // searching
    while (x != NULL) {
        y = x;

        if (z->data < x->data) {
            x = x->left;
        } else {
            x = x->right;
        }
    }

    // inserting
    if (y == NULL) {
        Tree = z; // 1st node
    } else if (z->data < y->data) {
        y->left = z;
    } else {
```

```
        y->right = z;
    }

    return Tree;
}

Nodeptr createBST() {
    int choice = 1;
    Nodeptr Tree = NULL;

    while(choice) {
        Nodeptr z = getnode();
        printf("Enter data: ");
        scanf("%d", &z->data);
        z->left = NULL, z->right = NULL;

        Tree = InsertBST(Tree, z);

        // exiting loop
        printf("Press 1 to enter next data or 0 to exit: ");
        scanf("%d", &choice);
        // printf("%d", choice)
    }

    return Tree;
}

/*
void deleteBST(Nodeptr Tree, int element) {
    Nodeptr x = Tree;
    Nodeptr y = NULL;

    // searching
    while (x != NULL) {
        y = x;

        if (element < x->data) {
            x = x->left;
        } else if (element < x->data) {
                    x = x->left;
                } else if (element != x->data) {
                        printf("Element: %d not found in Tree\n", element);
                        return 0;
                    }


        // Case I
        if (x->left )
```

```
    // Nodeptr y = NULL;
    // Nodeptr x = Tree;

    while (x != NULL) {
        y = x;

        if (z->data)
    }


}
*/

/*
inode(Nodeptr Tree) {
    if (Tree != NULL) {
        inode
    }
}
*/

void printBST_LVR(Nodeptr Tree) {
    // using LVR
    if (Tree != NULL) {
        printBST_LVR(Tree->left);
        printf("%d ", Tree->data);
        printBST_LVR(Tree->right);
    }
}


void printBST_LRV(Nodeptr Tree) {
    // using LRV
    if (Tree != NULL) {
        printBST_LRV(Tree->left);
        printBST_LRV(Tree->right);
        printf("%d ", Tree->data);
    }
}

void printBST_VLR(Nodeptr Tree) {
    // using VLR
    if (Tree != NULL) {
        printf("%d ", Tree->data);
        printBST_VLR(Tree->left);
```

```
        printBST_VLR(Tree->right);
    }
}

/*
Nodeptr searchBST(Nodeptr Tree, int z) {
    Nodeptr y = NULL;
    Nodeptr x = Tree;

    // searching
    while (x != NULL && x->data != z->data) {
        y = x;

        if (z < x->data) {
            x = x->left;
        } else {
            x = x->right;
        }
    }

    if (NULL != x && z == x->data){
        return x;
    } else {
        return NULL;
    }
}
*/

Nodeptr searchBST(Nodeptr T, int z) {
    Nodeptr x = T;
    while (x != NULL && x->data != z) {
        if (z < x->data) {
            x = x->left;
        } else {
            x = x->right;
        }
    }

    return x;
}

Nodeptr minimum(Nodeptr T) {
    Nodeptr x = T;

    while (x->left != NULL) {
        x = x->left;
    }
```

```
    return x;
}

Nodeptr maximum(Nodeptr T, int* counter) {
    Nodeptr x = T;

    while (x->right != NULL) {
        x = x->right;
        (*counter)++;
        printf("DEBUG: counter value %d\n", *counter);
    }

    return x;
}

Nodeptr inorderSuccessor(Nodeptr Tree, int z) {
    // search element in Tree firstly
    Nodeptr k = searchBST(Tree, z);

/*
    if (k != NULL){
        printf("DEBUG: searched element %d\n", k->data);
    } else {
        printf("DEBUG: node not found...\n");
    }
*/

    if (k == NULL) {
        printf("Node not found!!\n");
        return k;
    } else {
        if (k->right != NULL) {
            Nodeptr m = minimum(k->right);
            return m;
        } else {
            Nodeptr succ = NULL;
            Nodeptr T1 = Tree;

            while (T1 != k) {
                if (z < T1->data) {
                    succ = T1;
                    T1 = T1->left;  // update traversal pointer
                } else {
                    T1 = T1->right;
                }
            }
            return succ;
        }
```

```
    }

}

/* --- Added missing functions with minimal modifications --- */

// deleteBST: Deletes a node with the given key from the BST and returns the
new root.
Nodeptr deleteBST(Nodeptr root, int key) {
    if (root == NULL)
        return NULL;
    if (key < root->data)
        root->left = deleteBST(root->left, key);
    else if (key > root->data)
        root->right = deleteBST(root->right, key);
    else {
        // Node found
        if (root->left == NULL) {
            Nodeptr temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Nodeptr temp = root->left;
            free(root);
            return temp;
        } else {
            // Node with two children: get inorder successor
            Nodeptr temp = minimum(root->right);
            root->data = temp->data;
            root->right = deleteBST(root->right, temp->data);
        }
    }
    return root;
}

// inode: Inorder traversal function (similar to printBST_LVR)
void inode(Nodeptr Tree) {
    if (Tree != NULL) {
        inode(Tree->left);
        printf("%d ", Tree->data);
        inode(Tree->right);
    }
}
/* --- End of added functions --- */

int main() {
    Nodeptr Tree = createBST();
    printf("\n");
```

```
    printf("Tree in LVR: ");
    printBST_LVR(Tree);
    printf("\n");

    printf("Tree in LRV: ");
    printBST_LRV(Tree);
    printf("\n");

    printf("Tree in VLR: ");
    printBST_VLR(Tree);
    printf("\n");

    // Example deletion
    int element;
    printf("Enter element to delete: ");
    scanf("%d", &element);
    Tree = deleteBST(Tree, element);
    printf("Tree in LVR after deletion: ");
    printBST_LVR(Tree);
    printf("\n");

    int searching_element;
    printf("Enter element to search: ");
    scanf("%d", &searching_element);

    Nodeptr in_succ = inorderSuccessor(Tree, searching_element);
    if (in_succ != NULL)
        printf("Inorder successor of %d is %d\n", searching_element, in_succ-
>data);
    else
        printf("No inorder successor found for %d\n", searching_element);

    int counter = 0;
    Nodeptr max = maximum(Tree, &counter);
    printf("Maximum element %d, height: %d\n", max->data, counter);

    // Demonstrate inode (inorder traversal)
    printf("Inorder traversal using inode: ");
    inode(Tree);
    printf("\n");

    return 0;
}
```

OUTPUT :->

Enter data: 22

Press 1 to enter next data or 0 to exit: 1

Enter data: 43

Press 1 to enter next data or 0 to exit: 1

Enter data: 67

Press 1 to enter next data or 0 to exit: 1

Enter data: 43

Press 1 to enter next data or 0 to exit: 1

Enter data: 62

Press 1 to enter next data or 0 to exit: 0

Tree in LVR: 22 43 43 62 67

Tree in LRV: 62 43 67 43 22

Tree in VLR: 22 43 67 43 62

Enter element to delete: 43

Tree in LVR after deletion: 22 43 62 67

Enter element to search: 62

Inorder successor of 62 is 67

DEBUG: counter value 1

Maximum element 67, height: 1

Enter data: 34

Press 1 to enter next data or 0 to exit: 1

Enter data: 12

Press 1 to enter next data or 0 to exit: 1

Enter data: 78

Press 1 to enter next data or 0 to exit: 1

Enter data: 56

Press 1 to enter next data or 0 to exit: 1

Enter data: 56

Press 1 to enter next data or 0 to exit: 0

Tree in LVR: 12 34 56 56 78

Tree in LRV: 12 56 56 78 34

Tree in VLR: 34 12 78 56 56

Enter element to delete: 34

Tree in LVR after deletion: 12 56 56 78

Enter element to search: 21

Node not found!!

No inorder successor found for 21

DEBUG: counter value 1

Maximum element 78, height: 1

Inorder traversal using inode: 12 56 56 78

Assignment 9

Merge Sort

```c
#include <stdio.h>

void printArray(int arr[], int n){
    for (int i = 0; i < n ; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void merge(int arr[], int p, int q, int r) {
    int n1 = q - p + 1;
    int n2 = r - q;

    // decalre sub arrays
    int left[n1];  // frst sub arr
    int right[n2];  // 2nd sub array

    for (int i = 0; i < n1; i++) {
        left[i] = arr[p + i];
    }

    for (int j = 0 ; j < n2; j++) {
        right[j] = arr[q + 1 + j];
    }

    int i = 0, j = 0, k = p;
    while (i < n1 && j < n2) {
        if (left[i] <= right[j]) {
            arr[k] = left[i];
            i++;
        } else {
            arr[k] = right[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = left[i];
        k++;
        i++;
    }
```

```
    while (j < n2) {
        arr[k] = right[j];
        k++;
        j++;
    }
}


void mergeSort(int arr[], int p, int r) {
    if (p < r) {
        int q = (p + r) / 2; // mid-division

        mergeSort(arr, p, q);
        mergeSort(arr, q+1, r);
        merge(arr, p, q, r);
    }
}


int main() {
    int n;
    printf("Enter no. of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the list of numbers: ");
    for (int i = 0; i < n ; i++) {
        scanf("%d", &arr[i]);
    }

/*
    int n = 5;
    int arr[5] = {5,2,1,6,3};
*/
    printf("Original array: ");
    printArray(arr, n);

    mergeSort(arr, 0, n-1);

    printf("Sorted   array: ");
    printArray(arr, n);
}
```

OUTPUT :->

Enter no. of elements: 5

Enter the list of numbers: 12 43 78 12 3

Original array: 12 43 78 12 3

Sorted array: 3 12 12 43 78

Assignment 10

Quick Sort

```c
#include <stdio.h>
#include <stdlib.h>

void swap(int *e, int *f) {
    int x = *e;
    *e = *f;
    *f = x;
}

void printArray(int arr[], int n){
    for (int i = 0; i < n ; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i = i+1;

            swap(&arr[i], &arr[j]);
/*
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
```

```
*/
        }
    }

    swap(&arr[i+1], &arr[high]);
    return i+1;

}


void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        quickSort(arr, low, p-1);
        quickSort(arr, p+1, high);
    }
}

int main() {

    int n;
    printf("Enter no. of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the list of numbers: ");
    for (int i = 0; i < n ; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    printArray(arr, n);

    quickSort(arr, 0, n-1);

    printf("Sorted   array: ");
    printArray(arr, n);

}
```

OUTPUT :->

Enter no. of elements: 5

Enter the list of numbers: 56 12 43 875 12

Original array: 56 12 43 875 12

Sorted array: 12 12 43 56 875

Assignment 11

Radix Sort

```c
#include <stdio.h>

void radixSort(int arr[], int n) {
    // Finding max value
    int maxval = 0;

    for (int i = 0; i < n-1; i++) {
        if (arr[i] > maxval) {
            maxval = arr[i];
        }
    }

    int digit_pos = 1, max_units = 10;
    while (maxval / digit_pos > 0) {
        // 2D pocket array zero
        int pocket[10][max_units];
/*
        not necessary to zeo init

        for (int k = 0; k< 10; k++) {
            for (int k0 = 0; k0 < max_units ; k0++) {
                pocket[k][k0] = 0;
            }
        }
*/


        // Call index zero array
        int call_index[10];
        for (int ind = 0; ind < 10 ; ind++) {
            call_index[ind] = 0;
        }

        for (int i = 0; i < n; i++) {
            int k = (arr[i] / digit_pos) % 10;
            pocket[k][call_index[k]] = arr[i];
            call_index[k]++;

        }

        // collect all pockets values
        int index = 0;
```

```
        for (int j = 0; j < 10 ; j++) {


            if (call_index[j]) {
                for (int k = 0; k < call_index[j]; k++) {
                    arr[index] = pocket[j][k];
                    index++;
                }
            }
        }

        digit_pos *= 10;
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    radixSort(arr, n);

    printf("Sorted array: : ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

OUTPUT :-?

Enter number of elements: 6

Enter elements: 34 783 46 235 47 12

Sorted array: : 12 34 46 47 235 783

Assignment 12

Heap Sort

```c
#include <stdio.h>


void printArray(int arr[], int n) {
    printf("Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void swap(int *a, int *b) {
    int x = *a;
    *a = *b;
    *b = x;
}

void createHeap(int arr[], int n, int i) {
    int largest = i; // largest
    int l = 2*i + 1; // left child
    int r = 2*i + 2; // right child

    // check left child
    if (l < n && arr[l] > arr[largest]) {
        largest = l;
    }

    // check right child
    if (r < n && arr[r] > arr[largest]) {
        largest = r;
    }


    // swap if root isn't largest
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        // printArray(arr, n);
        createHeap(arr, n, largest);
    }
}


void heapSort(int arr[], int n) {
```

```
    int i = n/2 -1;

    while (i >= 0) {
        createHeap(arr, n, i);
        i--;
    }

    i = n -1;
    while (i >= 0) {
        swap(&arr[0], &arr[i]);
        createHeap(arr, i , 0);
        i--;
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    heapSort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

OUTPUT :->

Enter number of elements: 6

Enter elements: 673 23 12 263 124 2

Sorted array: 2 12 23 124 263 673

Assignment 13

Infix to postfix expression

```c
#include <stdio.h>

#define MAXSIZE 40


int priority(char x) {
    switch(x) {
        case '^':
            return 4;

        case '*':
        case '/':
            return 3;

        case '+':
        case '-':
            return 2;

        case '(':
            return 0;

        default:
            return -1;
    }
}


int is_operand(char opr){
    if ( (opr >= 'a' && opr <= 'z') || (opr >= 'A' && opr <= 'Z') || (opr >= '0' && opr <= '9') ) {
        return 1;
    } else return 0;
}


void push(char stack[], int* top, int maxsize, char ch){
    if (!(*top >= maxsize - 1)) {
        stack[++(*top)] = ch;
    } else {
        printf("Stack overflow !!!\n");
        return;
    }
}
```

```c
char pop(char stack[], int* top){
    if (*top < 0){
        printf("Stack underflow !!!\n");
        return '\0';
    }

    // (*top)--;
    return stack[(*top)--]; // return popped element
}


void in_to_post(char expr[], char post[]) {
    printf("Expression: %s\n", expr);

    char stack[MAXSIZE]; // intermediate array as stack
    int top = -1, post_index = 0;

    for (int i = 0; expr[i] != '\0'; i++) {
        if (is_operand(expr[i])) {
            // printf("%c", expr[i]);
            post[post_index++] = expr[i];

        } else if (expr[i] == '(') {
            // opening paranthesis case
            push(stack, &top, MAXSIZE, expr[i]);

        } else if (expr[i] == ')') {
            // closing paranthesis, then pop till opening comes in
            while (stack[top] != '(') {
                char y = pop(stack, &top);
                // printf("%c", y);
                post[post_index++] = y;
            }

            // pop out the '(' from stack before proceeding
            pop(stack, &top);

        } else { // other operator handling
            while (priority(stack[top]) >= priority(expr[i])) {
                char y = pop(stack, &top);
                // printf("%c", y); // pop & print until stack[top] has lower
prior then expr[i]
                post[post_index++] = y;
            }
            push(stack, &top, MAXSIZE, expr[i]);
        }
    }
```

```
    // remaining operators if any
    while (top > -1) {
        char y = pop(stack, &top);
        // printf("%c", y);
        post[post_index++] = y;
    }

    // printf("\n");

/*
    printf("Expression line by line: \n");
    int i = 0;
    while (expr[i] != '\0') {
        printf("%c\n", expr[i++]);
    }
*/
}

int main() {
    char expr[MAXSIZE], post[MAXSIZE];
    printf("Enter expression: ");
    scanf("%[^\n]", expr);
    in_to_post(expr, post);

    printf("Postfix expression: %s\n", post);
    char expr_reverse[MAXSIZE];


/*
    int top = -1;
    for (int i = 0; expr[i] != '\0'; i++, top++);
    printf("Top: %d\n", top);

    // int top = -1;
    for (int i = 0; expr[i] != '\0'; i++) {
        printf("Before: %d:-> %c, ", i, expr[i]);
        printf("Popped element: %c\n", pop(expr, &top));
    }
*/
}
```

OUTPUT :->

Enter expression: A+(B-C+D/E*(F+H-J)*K-L+N)-R

Expression: A+(B-C+D/E*(F+H-J)*K-L+N)-R

Postfix expression: ABC-DE/FH+J-*K*+L-N++R-

Assignment 14

Bubble Sort

```c
#include <stdio.h>
int main() {
int n, t;
printf("Enter size of array: ");
scanf("%d", &n);
int arr[n];
// Input array elements
for (int i = 0; i < n; i++) {
printf("Enter element no. %d: ", i + 1);
scanf("%d", &arr[i]);
}
for (int i = 0; i < n - 1; i++) {
for (int j = i + 1; j < n; j++) {
if (arr[i] > arr[j]) {
// Swap arr[i] and arr[j]
t = arr[i];
arr[i] = arr[j];
arr[j] = t;
}
}
}
printf("\nSorted list:\n");
// Print out sorted array
for (int i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
printf("\n");
return 0;
}
```

OUTPUT :->

Enter size of array: 6

Enter element no. 1: 23

Enter element no. 2: 12

Enter element no. 3: 43

Enter element no. 4: 12

Enter element no. 5: 4

Enter element no. 6: 23

Sorted list:

4 12 12 23 23 43